

10. Spring Boot Study Guide

이 문서는 현재 파일 공유 API 프로젝트를 예시로 Spring Boot의 기본 구조와 역할을 학습하기 위한 교보재다.

1. 이 프로젝트를 Spring Boot 관점에서 보기

이 프로젝트는 사용자가 회원가입과 로그인을 하고, 인증된 사용자가 파일을 업로드, 조회, 다운로드, 삭제할 수 있는 REST API 서버다.

Spring Boot 애플리케이션은 다음 흐름으로 동작한다.

```
HTTP 요청
-> Controller
-> Service
-> Repository 또는 Infra
-> DB, Redis, MinIO, 로컬 파일 시스템
-> 응답
```

예를 들어 파일 업로드 요청은 다음 순서로 처리된다.

```
POST /api/files
-> FileController.upload()
-> FileService.upload()
-> FileStorage.store()
-> FileMetadataRepository.save()
-> FileMetadataCache.put()
-> ApiResponse<FileResponse>
```

각 계층은 역할이 다르다. Controller는 HTTP 요청과 응답을 담당하고, Service는 비즈니스 규칙을 처리하고, Repository는 데이터 저장소와 연결한다. Infra 계층은 MinIO, 로컬 파일 시스템 같은 외부 기술을 감싼다.

2. 애플리케이션 시작점

파일:

```
backend/src/main/java/com/example/fileshare/FileShareApiApplication.java
```

Spring Boot 애플리케이션의 시작점이다. 일반적으로 다음 형태를 가진다.

```
@SpringBootApplication
public class FileShareApiApplication {

    public static void main(String[] args) {
        SpringApplication.run(FileShareApiApplication.class, args);
    }
}
```

@SpringBootApplication은 Spring Boot 설정의 출발점이다. 이 어노테이션을 기준으로 하위 패키지의 @RestController, @Service, @Repository, @Component, @Configuration 등을 찾아 Spring Bean으로 등록한다.

이 프로젝트의 기준 패키지는 다음과 같다.

```
com.example.fileshare
```

따라서 다음 패키지들이 자동 스캔 대상이 된다.

```
com.example.fileshare.auth
com.example.fileshare.user
com.example.fileshare.file
com.example.fileshare.common
com.example.fileshare.config
com.example.fileshare.infra
```

3. 패키지 구조와 책임

현재 백엔드 패키지는 기능과 책임 기준으로 나뉘어 있다.

```
auth/ 인증, 회원가입, 로그인
user/ 사용자 조회와 수정
file/ 파일 업로드, 조회, 다운로드, 삭제
common/ 공통 응답, 공통 에러, 예외 처리
config/ 보안 설정, JWT 필터
infra/ 외부 저장소 연동
```

각 기능 패키지 안에서는 다시 계층을 나눈다.

```
api/ HTTP API 계층
application/ 서비스 계층
domain/ 도메인 모델
repository/ 저장소 인터페이스와 구현체
```

예를 들어 파일 기능은 다음처럼 구성되어 있다.

```
file/api/FileController.java
file/application/FileService.java
file/domain/FileMetadata.java
file/repository/FileMetadataRepository.java
file/repository/InMemoryFileMetadataRepository.java
file/repository/jpa/JpaFileMetadataRepositoryAdapter.java
```

이 구조의 목적은 HTTP, 비즈니스 규칙, 저장소 구현을 분리하는 것이다.

4. Controller: HTTP 요청과 응답 담당

Controller는 클라이언트가 호출하는 API endpoint를 정의한다.

예시 파일:

```
backend/src/main/java/com/example/fileshare/auth/api/AuthController.java
backend/src/main/java/com/example/fileshare/file/api/FileController.java
backend/src/main/java/com/example/fileshare/user/api/UserController.java
```

AuthController는 /api/auth 하위 API를 담당한다.

```
@RestController
@RequestMapping("/api/auth")
public class AuthController {

    private final AuthService authService;

    public AuthController(AuthService authService) {
        this.authService = authService;
    }

    @PostMapping("/signup")
    public ApiResponse<UserResponse> signup(@Valid @RequestBody SignupRequest request) {
        return ApiResponse.ok(UserResponse.from(authService.signup(request)), "User
        created");
    }
}
```

중요한 포인트:

- @RestController: JSON 응답을 반환하는 웹 컨트롤러로 등록한다.
- @RequestMapping("/api/auth"): 이 컨트롤러의 공통 URL prefix를 지정한다.
- @PostMapping("/signup"): POST /api/auth/signup 요청을 처리한다.
- @RequestBody: HTTP body의 JSON을 Java 객체로 바꾼다.
- @Valid: 요청 DTO의 validation 규칙을 검사한다.

Controller는 직접 DB에 접근하지 않는다. 요청을 검증 가능한 객체로 받고, Service를 호출하고, 응답 DTO로 변환한다.

5. DTO: 요청과 응답 모델

DTO는 API 요청과 응답의 모양을 정의한다.

예시 파일:

```
auth/api/SignupRequest.java
auth/api/LoginRequest.java
auth/api/LoginResponse.java
user/api/UserResponse.java
file/api/FileResponse.java
```

DTO를 쓰는 이유는 API 계약과 내부 도메인 모델을 분리하기 위해서다.

예를 들어 User 도메인에는 passwordHash가 있지만, 응답 DTO인 UserResponse에는 비밀번호 해시를 포함하지 않는다. 이렇게 하면 내부 데이터 구조가 그대로 외부에 노출되는 것을 막을 수 있다.

6. Service: 비즈니스 규칙 담당

Service는 애플리케이션의 핵심 흐름을 처리한다.

예시 파일:

```
auth/application/AuthService.java
user/application/UserService.java
file/application/FileService.java
```

AuthService.signup()은 회원가입 규칙을 처리한다.

```
public User signup(SignupRequest request) {
    if (userRepository.existsByEmail(request.email())) {
        throw new BusinessException(ErrorCode.AUTH_DUPLICATED_EMAIL);
    }

    Instant now = Instant.now();
    User user = new User(
        null,
        request.email(),
        passwordEncoder.encode(request.password()),
        request.name(),
        UserRole.USER,
        now,
        now
    );
    return userRepository.save(user);
}
```

이 메서드는 다음 일을 한다.

- 이메일 중복 확인
- > 비밀번호 해시 처리
- > 기본 권한 USER 부여
- > 생성 시각 설정
- > 사용자 저장

FileService.upload()은 파일 업로드 규칙을 처리한다.

- 파일 크기와 확장자 검증
- > 파일 저장소에 실제 파일 저장
- > 파일 메타데이터 생성
- > DB에 메타데이터 저장
- > Redis 캐시에 메타데이터 저장

Service의 핵심 원칙은 "기능의 의미 있는 규칙은 Service에 둔다"는 것이다. Controller는 HTTP 처리에 집중하고, Repository는 저장소 접근에 집중한다.

7. Domain: 핵심 데이터 모델

Domain은 애플리케이션이 다루는 핵심 개념을 표현한다.

예시 파일:

```
user/domain/User.java
user/domain/UserRole.java
file/domain/FileMetadata.java
file/domain/FileStatus.java
file/domain/ScanStatus.java
```

User는 사용자 자체를 표현하고, FileMetadata는 업로드된 파일의 메타데이터를 표현한다.

이 프로젝트에서는 JPA Entity와 Domain을 분리했다.

Domain: 서비스 계층이 사용하는 핵심 모델
JPA Entity: DB 테이블과 매핑되는 영속성 모델

분리한 이유는 DB 구조가 바뀌어도 서비스 계층의 모델을 최대한 안정적으로 유지하기 위해서다.

8. Repository: 저장소 접근 추상화

Repository는 데이터를 저장하고 조회하는 역할을 한다.

예시 인터페이스:

```
user/repository/UserRepository.java
file/repository/FileMetadataRepository.java
```

FileMetadataRepository는 다음 기능을 정의한다.

```
public interface FileMetadataRepository {

    FileMetadata save(FileMetadata fileMetadata);

    Optional<FileMetadata> findById(Long id);

    List<FileMetadata> findVisibleByOwnerId(Long ownerId);
}
```

Service는 이 인터페이스에만 의존한다. 실제 구현은 실행 프로필에 따라 바뀐다.

기본 실행
-> InMemoryFileMetadataRepository

postgres 프로필
-> JpaFileMetadataRepositoryAdapter

이 구조 덕분에 FileService는 저장소가 메모리인지 PostgreSQL인지 몰라도 된다.

9. JPA: Java 객체와 DB 테이블 연결

JPA는 Java 객체를 DB 테이블에 저장하고 조회할 수 있게 해주는 기술이다.

예시 파일:

```
user/repository/jpa/UserEntity.java
user/repository/jpa/SpringDataUserJpaRepository.java
user/repository/jpa/JpaUserRepositoryAdapter.java
file/repository/jpa/FileMetadataEntity.java
file/repository/jpa/SpringDataFileMetadataJpaRepository.java
file/repository/jpa/JpaFileMetadataRepositoryAdapter.java
```

UserEntity는 users 테이블과 매핑된다.

```
@Entity
@Table(name = "users")
public class UserEntity {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true, length = 255)
    private String email;
```

```
}
```

중요한 포인트:

- @Entity: JPA가 관리하는 DB 매핑 객체다.
- @Table(name = "users"): DB 테이블 이름을 지정한다.
- @Id: 기본 키다.
- @GeneratedValue: DB가 ID를 자동 생성한다.
- @Column: 컬럼 제약을 지정한다.

Spring Data JPA Repository는 기본 CRUD를 자동으로 제공한다.

```
public interface SpringDataUserJpaRepository extends JpaRepository<UserEntity, Long> {  
    Optional<UserEntity> findByEmail(String email);  
    boolean existsByEmail(String email);  
}
```

findByEmail처럼 메서드 이름을 규칙에 맞게 작성하면 Spring Data JPA가 쿼리를 만들어준다.

10. Adapter: 도메인 Repository와 JPA 연결

이 프로젝트에서 Service는 UserRepository에 의존한다. 그런데 Spring Data JPA는 SpringDataUserJpaRepository를 제공한다. 둘을 연결하는 클래스가 Adapter다.

예시:

```
@Repository  
@Profile("postgres")  
public class JpaUserRepositoryAdapter implements UserRepository {  
    private final SpringDataUserJpaRepository jpaRepository;  
  
    @Override  
    public User save(User user) {  
        return jpaRepository.save(UserEntity.from(user)).toDomain();  
    }  
}
```

Adapter의 역할:

```
Domain User  
-> UserEntity  
-> JPA save  
-> UserEntity  
-> Domain User
```

이 구조는 "서비스 계층이 JPA를 직접 알지 않게" 만든다.

11. Profile: 실행 환경별 스위치

Spring Profile은 실행 모드에 따라 설정과 Bean을 바꾸는 기능이다.

기본 실행:

```
./gradlew bootRun
```

이 경우 인메모리 저장소와 로컬 파일 저장소를 사용한다.

외부 저장소 실행:

```
SPRING_PROFILES_ACTIVE=postgres,redis,minio ./gradlew bootRun
```

이 경우 다음 Bean과 설정이 활성화된다.

postgres -> JPA/PostgreSQL 저장소
redis -> Redis 캐시
minio -> MinIO 파일 저장소

프로필별 설정 파일:

application-postgres.yml
application-redis.yml
application-minio.yml

프로필별 Bean 예시:

```
@Profile("postgres")
public class JpaUserRepositoryAdapter implements UserRepository {
}

@Profile("!redis")
public class NoOpFileMetadataCache implements FileMetadataCache {
}
```

!redis는 Redis 프로필이 꺼져 있을 때 활성화된다는 뜻이다.

12. Redis: 캐시 저장소

Redis는 이 프로젝트에서 원본 저장소가 아니라 캐시로 사용된다.

예시 파일:

file/application/FileMetadataCache.java
file/application/RedisFileMetadataCache.java
file/application/NoOpFileMetadataCache.java

캐시 인터페이스:

```
public interface FileMetadataCache {

    Optional<FileMetadata> findById(Long id);

    void put(FileMetadata fileMetadata);

    void evict(Long id);
}
```

Redis 구현체는 다음 키 형식으로 파일 메타데이터를 저장한다.

files:metadata:{fileId}

TTL은 5분이다.

```
private static final Duration TTL = Duration.ofMinutes(5);
private static final String KEY_PREFIX = "files:metadata:";
```

파일 단건 조회 흐름:

Redis 캐시 조회
-> 캐시에 있으면 반환
-> 캐시에 없으면 PostgreSQL 조회
-> 조회 결과를 Redis에 다시 저장

Redis가 꺼져 있을 때는 NoOpFileMetadataCache가 사용된다. 이 구현체는 아무것도 저장하지 않고 항상 cache miss를 반환한다.

13. MinIO: 실제 파일 저장소

MinIO는 이 프로젝트에서 실제 파일 바이너리를 저장한다.

예시 파일:

infra/storage/FileStorage.java
infra/storage/LocalFileStorage.java
infra/storage/MinioFileStorage.java

FileStorage 인터페이스:

```
public interface FileStorage {  
    StoredFile store(String originalFileName, InputStream inputStream) throws IOException;  
    StorageObject load(String storagePath) throws IOException;  
}
```

기본 실행에서는 LocalFileStorage가 사용되고, minio 프로파일에서는 MinioFileStorage가 사용된다.

파일 업로드 시 저장 위치:

실제 파일 내용
-> MinIO bucket

파일명, 크기, ownerId, storagePath
-> PostgreSQL files 테이블

중요한 연결 고리는 files.storage_path다. 이 값이 MinIO object key와 같다.

PostgreSQL files.storage_path
-> MinIO object key
-> 실제 파일 다운로드

14. Security: 인증과 인가

보안 설정 파일:

config/security/SecurityConfig.java
config/security/JwtAuthenticationFilter.java
config/security/JwtTokenProvider.java
config/security/AuthUserPrincipal.java

SecurityConfig는 어떤 API를 공개하고 어떤 API에 인증을 요구할지 정한다.

```
.authorizeHttpRequests(auth -> auth  
    .requestMatchers("/api/health", "/actuator/health", "/api/auth/signup",  
        "/api/auth/login", "/api/auth/refresh").permitAll()  
    .anyRequest().authenticated()  
)
```

의미:

/api/health, /api/auth/signup, /api/auth/login
-> 인증 없이 접근 가능

그 외 API
-> JWT 인증 필요

JWT 인증 흐름:

로그인 성공
-> JwtTokenProvider가 access token 발급
-> 클라이언트가 Authorization: Bearer {token} 전송
-> JwtAuthenticationFilter가 토큰 검증
-> AuthUserPrincipal을 SecurityContext에 저장
-> Controller에서 @AuthenticationPrincipal로 현재 사용자 사용

파일 API는 현재 사용자를 이렇게 받는다.

```
public ApiResponse<FileResponse> upload(  
    @AuthenticationPrincipal AuthUserPrincipal principal,  
    @RequestParam MultipartFile file  
)
```

15. Exception Handling: 예외 응답 통일

예외 처리 파일:

common/exception/BusinessException.java
common/exception/GlobalExceptionHandler.java
common/error/ErrorCode.java
common/api/ApiResponse.java

Service에서 문제가 생기면 BusinessException을 던진다.

```
throw new BusinessException(ErrorCode.AUTH_DUPLICATED_EMAIL);
```

GlobalExceptionHandler가 이 예외를 잡아 공통 응답 형태로 변환한다.

```
@ExceptionHandler(BusinessException.class)
public ResponseEntity<ApiResponse<Void>> handleBusinessException(BusinessException
exception) {
    ErrorCode errorCode = exception.errorCode();
    return ResponseEntity
        .status(errorCode.status())
        .body(ApiResponse.fail(errorCode.name(), exception.getMessage()));
}
```

이 구조의 장점은 API 에러 응답이 일관된다는 것이다.

예시:

```
{
  "success": false,
  "message": "Resource not found",
  "errorCode": "RESOURCE_NOT_FOUND"
}
```

16. Configuration: 설정과 Bean 등록

Configuration은 Spring Bean을 직접 등록하거나 프레임워크 설정을 구성할 때 사용한다.

예시:

```
config/security/SecurityConfig.java
```

비밀번호 암호화 객체는 @Bean으로 등록한다.

```
@Bean
public PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder();
}
```

이렇게 등록된 PasswordEncoder는 AuthService 생성자에 자동 주입된다.

```
public AuthService(UserRepository userRepository, PasswordEncoder passwordEncoder,
    JwtTokenProvider jwtTokenProvider) {
    this.userRepository = userRepository;
    this.passwordEncoder = passwordEncoder;
    this.jwtTokenProvider = jwtTokenProvider;
}
```

17. Dependency Injection: 의존성 주입

Spring은 필요한 객체를 직접 생성해서 연결해준다. 이것을 의존성 주입이라고 한다.

예시:

```
public FileService(
    FileMetadataRepository fileMetadataRepository,
    FileStorage fileStorage,
    FileMetadataCache fileMetadataCache
){
    this.fileMetadataRepository = fileMetadataRepository;
    this.fileStorage = fileStorage;
    this.fileMetadataCache = fileMetadataCache;
}
```

FileService는 구현체를 직접 만들지 않는다.

```
new JpaFileMetadataRepositoryAdapter()
new MinioFileStorage()
new RedisFileMetadataCache()
```

이런 코드를 Service 안에 쓰지 않는다. Spring이 현재 프로필과 설정에 맞는 Bean을 찾아 넣어준다.

이 방식의 장점:

- Service 테스트가 쉬워진다.
- 저장소 구현을 바꿔도 Service 코드를 유지할 수 있다.
- 로컬 개발, 테스트, 운영 환경을 프로파일로 나눌 수 있다.

18. API 요청 처리 예시

회원가입 흐름:

```
POST /api/auth/signup
-> AuthController.signup()
-> AuthService.signup()
-> UserRepository.existsByEmail()
-> PasswordEncoder.encode()
-> UserRepository.save()
-> UserResponse.from()
-> ApiResponse.ok()
```

로그인 흐름:

```
POST /api/auth/login
-> AuthController.login()
-> AuthService.login()
-> UserRepository.findByEmail()
-> PasswordEncoder.matches()
-> JwtTokenProvider.generate()
-> LoginResponse
```

파일 업로드 흐름:

```
POST /api/files
-> JwtAuthenticationFilter
-> FileController.upload()
-> FileService.upload()
-> 확장자와 크기 검증
-> FileStorage.store()
-> FileMetadataRepository.save()
-> FileMetadataCache.put()
-> FileResponse
```

파일 단건 조회 흐름:

```
GET /api/files/{id}
-> JwtAuthenticationFilter
-> FileController.get()
-> FileService.get()
-> FileMetadataCache.findById()
-> FileMetadataRepository.findById()
-> 소유자 또는 ADMIN 권한 확인
-> FileResponse
```

파일 다운로드 흐름:

```
GET /api/files/{id}/download
-> FileService.download()
-> FileMetadata 조회
-> 파일 상태 확인
-> FileStorage.load()
-> ResponseEntity<Resource>
```

파일 삭제 흐름:

```
DELETE /api/files/{id}
-> FileService.delete()
-> FileMetadata 조회
-> 권한 확인
-> status를 DELETED로 변경
-> FileMetadataRepository.save()
-> FileMetadataCache.evict()
```

현재 삭제는 실제 파일을 지우는 물리 삭제가 아니라 메타데이터 상태를 바꾸는 논리 삭제다.

19. 테스트: 기능이 실제로 동작하는지 검증

예시 파일:

```
backend/src/test/java/com/example/fileshare/auth/AuthApiIntegrationTests.java
backend/src/test/java/com/example/fileshare/file/FileApiIntegrationTests.java
backend/src/test/java/com/example/fileshare/persistence/PostgresProfileIntegrationTests.java
```

PostgresProfileIntegrationTests는 postgres 프로파일에서 JPA 저장소가 동작하는지 확인한다.

```
@SpringBootTest(properties = {
    "spring.datasource.url=jdbc:h2:mem:fileshare-postgres-profile;MODE=PostgreSQL;DATABASE_TO_LOWER=TRUE;DB_CLOSE_DELAY=-1"
})
@AutoConfigureMockMvc
@ActiveProfiles("postgres")
class PostgresProfileIntegrationTests {
}
```

중요한 포인트:

- @SpringBootTest: Spring Boot 애플리케이션을 테스트 환경에서 띄운다.
- @AutoConfigureMockMvc: 실제 서버 포트 없이 HTTP 요청을 테스트한다.
- @ActiveProfiles("postgres"): 테스트에서 postgres 프로필을 켜다.
- H2 PostgreSQL compatibility mode: 실제 PostgreSQL 없이 JPA 매핑을 검증한다.

테스트 실행:

```
cd backend
./gradlew test
```

20. 현재 프로젝트로 Spring Boot를 공부하는 순서

처음부터 모든 파일을 동시에 보지 말고 요청 흐름 기준으로 읽는 것이 좋다.

1. 애플리케이션 시작점 보기

```
FileShareApiApplication.java
```

2. 인증 API 흐름 보기

```
AuthController.java
AuthService.java
UserRepository.java
InMemoryUserRepository.java
JpaUserRepositoryAdapter.java
```

3. 파일 API 흐름 보기

```
FileController.java
FileService.java
FileStorage.java
LocalFileStorage.java
MinioFileStorage.java
FileMetadataRepository.java
```

4. 공통 응답과 예외 처리 보기

```
ApiResponse.java
BusinessException.java
ErrorCode.java
GlobalExceptionHandler.java
```

5. 보안 흐름 보기

```
SecurityConfig.java
```

JwtAuthenticationFilter.java
JwtTokenProvider.java
AuthUserPrincipal.java

6. 프로필과 외부 저장소 보기

application.yml
application-postgres.yml
application-redis.yml
application-minio.yml

21. 핵심 개념 요약

Spring Boot 핵심 개념을 이 프로젝트에 매핑하면 다음과 같다.

Controller
-> HTTP endpoint 정의
-> AuthController, FileController, UserController

Service
-> 비즈니스 규칙 처리
-> AuthService, FileService, UserService

Repository
-> 데이터 저장과 조회 추상화
-> UserRepository, FileMetadataRepository

JPA Entity
-> DB 테이블과 Java 객체 매핑
-> UserEntity, FileMetadataEntity

Spring Data JPA Repository
-> CRUD와 쿼리 메서드 제공
-> SpringDataUserJpaRepository, SpringDataFileMetadataJpaRepository

Profile
-> 실행 환경별 Bean과 설정 선택
-> postgres, redis, minio

Security
-> 인증, 인가, JWT 검증
-> SecurityConfig, JwtAuthenticationFilter

Exception Handler
-> 예외 응답 통일
-> GlobalExceptionHandler

Infra
-> 외부 기술 연동
-> MinioFileStorage, LocalFileStorage, RedisFileMetadataCache

이 프로젝트의 좋은 학습 포인트는 같은 Service 코드가 프로필에 따라 다른 저장소 구현을 사용할 수 있다는 점이다.

기본 모드
-> 인메모리 Repository
-> 로컬 파일 저장소
-> NoOp 캐시

외부 저장소 모드
-> PostgreSQL JPA Repository
-> MinIO 파일 저장소
-> Redis 캐시

즉 Spring Boot는 Controller, Service, Repository를 정해진 역할로 나누고, Spring Container가 필요한 객체를 연결해주며, Profile과 Configuration으로 실행 환경을 전환할 수 있게 해주는 프레임워크라고 볼 수 있다.